Chapter 9 — Computer Programming

Syllabus (Unit 9, Annexure-VII C, CEN 05/2025): Programming concepts; Fundamentals of 'C' and C++; Operators in 'C' and C++; Control Statements; Functions, Array, String & Pointers, File Structure; Data Structure and DBMS.

9.0 Weightage & Trend Box

Parameter	Detail
Weight in Technical section	**~10% → 5 questions out of 50** (observed in 22-04-2025 CBT-2 Electronics paper)
Difficulty level	Easy to Moderate — dominated by **"predict the output"** and **one-line definition** questions
Negative marking	1/3 mark per wrong answer
Most tested areas	Operators (especially **bitwise** and **unary**), control-statement outputs (**infinite loops**), basics of `main()`, ternary operator syntax

Actual questions observed in the 22-04-2025 CBT-2 paper (solved fully in the PYQ Corner, Section 9.14):

1. Which operator performs **bitwise OR** in C? → `|`
2. If a C program contains only a **single function**, that function **must be** `main()`.
3. Identify the **unary operator** among given options → `--` (decrement).
4. Correct syntax of the **conditional (ternary) operator** → `expr1 ? expr2 : expr3`.
5. Output of `while(i<=10) printf("%d",i);` **without increment** → **infinite loop printing 1** repeatedly.

Preparation strategy: This unit gives the *best marks-per-hour ratio* in the paper. Every year the questions come from (a) operator classification/precedence, (b) tracing tiny programs, (c) one-liner definitions from Data Structures and DBMS. Master the 15 solved output programs (Section 9.13) and the rapid-revision sheet (Section 9.16) and these 5 marks are secured.

9.1 Programming Concepts

9.1.1 Algorithm

An **algorithm** is a finite, ordered sequence of unambiguous steps to solve a problem. Properties:

- **Finiteness** — must terminate after a finite number of steps.
- **Definiteness** — every step precisely defined.
- **Input** — zero or more inputs.
- **Output** — at least one output.
- **Effectiveness** — each step basic enough to be carried out exactly.

9.1.2 Flowchart Symbols (exam favourite)

Symbol shape	Name	Purpose
Oval / rounded rectangle	Terminal	Start / Stop
Parallelogram	Input/Output	Read or print data
Rectangle	Process	Computation / assignment
Diamond (rhombus)	Decision	Condition test (yes/no branch)
Circle	Connector	Join flow lines (on same page)
Pentagon (home plate)	Off-page connector	Continue on another page
Arrow	Flow line	Direction of control
Rectangle with double side bars	Predefined process	Subroutine / function call

9.1.3 Pseudocode

Pseudocode is an informal, English-like description of an algorithm using programming-style constructs (`IF...THEN...ELSE` , `WHILE` , `FOR`) without strict syntax. It is language-independent and easier to convert into any programming language.

9.1.4 Language Translators — Compiler vs Interpreter vs Assembler

Feature	Compiler	Interpreter	Assembler
Translates	Entire high-level program at once	High-level program line by line	Assembly language → machine code
Output	Object/machine code file	No object file; executes directly	Machine code
Speed of execution	Fast (after compilation)	Slow	Fast
Error reporting	All errors after full scan	Stops at first error, line by line	Line by line
Examples	C, C++	Python, BASIC (classic), JavaScript	MASM, TASM

- **Linker**: combines object files + library code into an executable.
- **Loader**: places the executable into memory for execution.
- **Preprocessor** (in C): processes `#` directives (`#include` , `#define`) before compilation.

9.1.5 Generations of Languages

1GL — Machine language (binary). 2GL — Assembly. 3GL — High-level procedural (C, Pascal). 4GL — Very high level / query oriented (SQL). 5GL — Constraint/AI based (Prolog).

9.1.6 Structured Programming vs Object-Oriented Programming

Structured (Procedural) — e.g. C	Object-Oriented — e.g. C++
Program divided into **functions**	Program divided into **objects** (instances of classes)
Top-down design	**Bottom-up** design
Data moves freely between functions (less secure)	Data hidden inside objects (**data hiding/encapsulation**)
No overloading	Function/operator **overloading** supported
Adding new data/functions is harder	Easy extension via **inheritance**
Focus on procedure/logic	Focus on data

9.2 Fundamentals of C

C was developed by **Dennis Ritchie** at **Bell Laboratories (AT&T) in 1972**. C++ was developed by **Bjarne Stroustrup** (1979–83, originally "C with Classes"). C is a **middle-level, procedural, compiled, case-sensitive** language.

9.2.1 Structure of a C Program

```
/* Documentation section  – comments              */
#include <stdio.h>       /* Link/preprocessor section */
#define PI 3.14159       /* Definition section        */
int g = 10;              /* Global declaration section */

int main(void)           /* main() – execution starts here */
{
    /* declaration part */
    int r = 2;
    /* executable part  */
    printf("Area = %f\n", PI * r * r);
    return 0;            /* status returned to OS */
}
/* user-defined functions (sub-program section) follow */
```

Key facts (repeated PYQ material):

- Execution of every C program **always begins at** `main()`.
- **If a C program contains only one function, that function must be** `main()` — a program without `main()` will compile-link fail. *(Asked verbatim in 2025.)*
- A C program can have **only one** `main()` but any number of other functions.
- Statements end with a **semicolon** `;`. C is **free-format** (whitespace insensitive).
- Comments: `/* ... */` (multi-line), `// ...` (single line, C99/C++).

9.2.2 Header Files

Header	Contains
`stdio.h`	`printf` , `scanf` , `fopen` , `fgetc` , `EOF`
`conio.h`	`clrscr()` , `getch()` (Turbo C, non-standard)
`string.h`	`strlen` , `strcpy` , `strcmp` , `strcat` , `strrev`
`math.h`	`sqrt` , `pow` , `abs(float→fabs)` , `sin`
`stdlib.h`	`malloc` , `calloc` , `free` , `exit` , `atoi` , `rand`
`ctype.h`	`isalpha` , `isdigit` , `toupper` , `tolower`

`#include <file>` searches standard directories; `#include "file"` searches the current directory first.

9.2.3 C Tokens

The smallest individual units of a program. Six types:

1. **Keywords** — reserved words; **C has 32 keywords** (ANSI C): `auto break case char const continue default do double else enum extern float for goto if int long register return short signed sizeof static struct switch typedef union unsigned void volatile while` . All keywords are in **lowercase**.
2. **Identifiers** — names of variables/functions/arrays. Rules: begin with a **letter or underscore**; subsequent characters letters/digits/underscore; **no spaces or special symbols**; cannot be a keyword; case-sensitive (`Sum` ≠ `sum`).
3. **Constants** — fixed values (integer, real, character `'A'` , string `"A"`).
4. **Strings** — sequence of characters in double quotes, terminated by `'\0'` .
5. **Operators** — see Section 9.3.
6. **Special symbols** — `[] {} () , ; # * &` etc.

9.2.4 Data Types and Typical Sizes

Type	Typical size (16-bit / 32-bit compiler)	Range (32-bit `int`)	Format specifier
`char`	1 byte / 1 byte	−128 to +127	`%c`
`unsigned char`	1 / 1	0 to 255	`%c`
`int`	2 / **4 bytes**	−2,147,483,648 to +2,147,483,647	`%d`
`unsigned int`	2 / 4	0 to 4,294,967,295	`%u`
`short int`	2 / 2	−32,768 to +32,767	`%hd`
`long int`	4 / 4 (or 8)	—	`%ld`
`float`	4 bytes	~6 significant digits	`%f`
`double`	8 bytes	~15 significant digits	`%lf`
`long double`	10/12/16 bytes	—	`%Lf`
`void`	— (no value)	—	—

Other specifiers: `%o` octal, `%x` hexadecimal, `%s` string, `%p` pointer address, `%e` exponential.

Type modifiers: `signed, unsigned, short, long`. **Escape sequences:** `\n` newline, `\t` tab, `\0` null, `\\` backslash, `\"` double quote, `\b` backspace, `\r` carriage return, `\a` alert(bell).

9.2.5 Constants

- **Literal constants**: `10`, `0x1A` (hex), `012` (octal), `3.14`, `'A'`, `"RRB"`.
- `const int x = 5;` — read-only variable.
- `#define MAX 100` — preprocessor macro (no memory, textual substitution, no semicolon).
- `enum color {RED, GREEN=5, BLUE};` — RED=0, GREEN=5, BLUE=6.

9.2.6 Storage Classes (very frequent MCQ)

Storage class	Storage	Default initial value	Scope	Lifetime
`auto`	Memory (stack)	**Garbage**	Local to block	Until block ends
`register`	**CPU register** (request)	Garbage	Local to block	Until block ends
`static`	Memory (data segment)	**Zero**	Local to block (but value **persists** between calls)	Entire program run
`extern`	Memory (data segment)	Zero	**Global** (declared elsewhere)	Entire program run

- Default storage class of a local variable = `auto`; of a global variable = `extern`-like (static duration).
- **Address of a `register` variable cannot be taken** (`&` not allowed).
- A `static` local variable is **initialised only once**.

9.2.7 Scope

- **Block scope** — variable visible only inside `{ }` where declared.
- **File (global) scope** — declared outside all functions; visible from declaration to end of file.
- **Function scope** — labels (used with `goto`).
- Local variable **shadows** a global of the same name inside its block.

9.3 Operators in C and C++ ★ (highest-yield section)

9.3.1 Classification by Number of Operands (asked in 2025)

Class	Operands	Examples			
Unary	1	`++ -- ! ~ sizeof & (address) * (dereference) unary + and - (type)cast`			
Binary	2	`+ - * / % < > == && \`	\	& \	^ << >> = +=`
Ternary	3	`? :` (the **only** ternary operator in C)			

> **2025 PYQ:** among `+ (binary), * (binary), && (binary), --`, the **unary operator is `--`**.

9.3.2 Arithmetic Operators

`+ - * / %`. Rules:

- `%` (modulus) works **only on integers** — `5.0 % 2` is a compile error (use `fmod`).
- **Integer division truncates**: `7/2 = 3`, but `7.0/2 = 3.5`.
- Sign of `a % b` follows the sign of `a` (C99): `-7 % 2 = -1`.

9.3.3 Relational Operators

`< <= > >= == !=` — result is **1 (true) or 0 (false)**, type `int`.

Classic trap: `=` (assignment) vs `==` (comparison). `if (x = 0)` assigns 0 → condition false.

9.3.4 Logical Operators

`&&` (AND), `||` (OR), `!` (NOT). Operate on truth values; any **non-zero value is true**.

Short-circuit evaluation: in `a && b`, if `a` is false, `b` is **not evaluated**; in `a || b`, if `a` is true, `b` is not evaluated. (Source of many output questions — see Program 6.)

9.3.5 Bitwise Operators ★ (2025 PYQ)

Operator	Name	Example (a=12=1100_2, b=10=1010_2)		
`&`	Bitwise AND	`a & b` = 1000_2 = **8**		
`\`	`` ` ``	**Bitwise OR**	`a \	b` = 1110_2 = **14**
`^`	Bitwise XOR	`a ^ b` = 0110_2 = **6**		
`~`	One's complement (unary)	`~a` = −(a+1) = **−13**		
`<<`	Left shift	`a << 2` = a × 2^2 = **48**		
`>>`	Right shift	`a >> 2` = a ÷ 2^2 = **3**		

> **2025 PYQ:** the operator that performs bitwise OR is `|` (not `||` which is *logical* OR, not `&` which is bitwise AND).

Shortcuts: `x << n` = $x \cdot 2^n$; `x >> n` = $x/2^n$ (integer); `x ^ x = 0`; `x ^ 0 = x`; swapping without temp uses XOR three times.

9.3.6 Assignment & Compound Assignment

`= += -= *= /= %= <<= >>= &= |= ^=`. `a += 5` ≡ `a = a + 5`. Assignment itself yields a value: `a = b = c = 10;` (right-to-left).

9.3.7 Increment/Decrement — Pre vs Post ★

Form	Meaning
`++a` (pre-increment)	Increment **first**, then use the value
`a++` (post-increment)	Use current value, **then** increment
`--a` / `a--`	Same logic with decrement

```
int a = 5, b;
b = a++;   /* b = 5, a = 6 */
b = ++a;   /* a = 7, b = 7 */
```

These operate only on **variables (lvalues)** — `5++` is illegal.

9.3.8 Conditional (Ternary) Operator ★ (2025 PYQ)

Syntax: `expression1 ? expression2 : expression3;`

If `expression1` is true (non-zero) → result is `expression2`, else `expression3`.

```
int max = (a > b) ? a : b;      /* max of two numbers */
printf("%d", (7 > 3) ? 10 : 20);  /* prints 10 */
```

It is the only operator in C taking **three operands**; it can replace a simple `if-else`. Nesting is allowed: `(a>b) ? ((a>c)?a:c) : ((b>c)?b:c)` gives max of three.

9.3.9 sizeof, Comma, and Other Operators

- `sizeof` — **compile-time unary operator**, returns size in **bytes**; `sizeof(int)` = 4 (32-bit), `sizeof('A')` = 4 in C (char literal is `int` in C; **1 in C++**).
- **Comma operator** `,` — evaluates left to right; value of the whole expression = **rightmost** operand: `x = (1, 2, 3);` → x = 3. Comma has the **lowest precedence** of all operators.
- `&` address-of, `*` dereference, `.` member access, `->` member via pointer, `[]` subscript, `()` function call, `(type)` cast.
- **C++ extras:** `::` scope resolution, `new`, `delete`, `>>` / `<<` overloaded as extraction/insertion for streams, `.*`, `->*`.

9.3.10 Precedence & Associativity Table (memorise top and bottom)

Rank (high→low)	Operators	Associativity
1	`() [] . -> postfix ++ --`	Left → Right
2	`prefix ++ -- unary + - ! ~ * & sizeof (cast)`	**Right → Left**
3	`* / %`	L → R
4	`+ -`	L → R
5	`<< >>`	L → R
6	`< <= > >=`	L → R
7	`== !=`	L → R
8	`&`	L → R
9	`^`	L → R
10	`|	L → R
11	`&&`	L → R
12	`||`	L → R
13	`? :`	**Right → Left**
14	`= += -= *= /= %= &= ^= |= <<= >>=`	**Right → Left**
15 (lowest)	`,`	L → R

Memory aid for 3–12: **A**rithmetic > **S**hift > **R**elational > **E**quality > **B**itwise (& ^ |) > **L**ogical (&& ||).

Worked example: `x = 2 + 3 * 4 > 10 && 5;` → `3*4=12` → `2+12=14` → `14>10=1` → `1&&5=1` → `x=1`.

9.4 Control Statements

9.4.1 Selection: if, if–else, nested if, else–if ladder

```
if (marks >= 90)      grade = 'A';
else if (marks >= 75) grade = 'B';
else                  grade = 'C';
```

- Condition non-zero ⇒ true. `if(1)` always executes; `if(0)` never.
- **Dangling else:** an `else` always pairs with the **nearest unmatched** `if`:

```
if (a > b)
    if (a > c) printf("A");
else printf("B");     /* this else belongs to if(a>c), NOT if(a>b) */
```

- **Semicolon trap:** `if (x > 5);` — the semicolon is an empty statement; the following block executes unconditionally.

9.4.2 switch–case and Fallthrough ★

```
switch (expression)   /* must be int or char type — NOT float or string */
{
    case 1: printf("One");   break;
    case 2: printf("Two");   break;
    default: printf("Other");
}
```

Rules:

- Case labels must be **integer/character constants** (constant expressions), all **distinct**; `float` and variables not allowed.
- **Without `break`, control falls through** into the next case — a classic output question:

```
int x = 2;
switch (x) {
    case 1: printf("A");
    case 2: printf("B");
    case 3: printf("C");
    default: printf("D");
}
/* Output: BCD  (fallthrough from case 2) */
```

- `default` is optional and may appear anywhere.

9.4.3 Loops

Loop	Type	Minimum executions
`while (cond) { }`	**Entry-controlled** (pre-test)	0
`do { } while (cond);`	**Exit-controlled** (post-test)	**1** (body runs at least once)
`for (init; cond; update)`	Entry-controlled	0

- `for(;;)` and `while(1)` are **infinite loops**.
- In `for`, all three expressions are optional; missing condition is treated as **true**.
- `do-while` ends with a **semicolon** after `while(cond)`.

9.4.4 break, continue, goto

- `break` — exits the **innermost** loop or switch immediately.
- `continue` — skips remaining body, jumps to the next iteration (in `for`, the update expression still executes).
- `goto label;` — unconditional jump to `label:` within the same function (discouraged; breaks structured flow).

9.4.5 Classic Trick Outputs (exam gold)

(a) Missing increment → infinite loop (2025 PYQ):

```
int i = 1;
while (i <= 10)
    printf("%d ", i);      /* i never changes */
/* Output: 1 1 1 1 ... printed infinitely (infinite loop) */
```

(b) Semicolon after loop:

```
int i;
for (i = 0; i < 5; i++);   /* empty body due to ; */
printf("%d", i);           /* Output: 5 */
```

(c) Post-decrement in while condition:

```
int i = 3;
while (i--)
    printf("%d ", i);      /* Output: 2 1 0 (condition uses old value) */
```

(d) do–while runs once even when false:

```
int i = 10;
do { printf("Hello"); } while (i < 5);   /* Output: Hello (printed once) */
```

9.5 Functions

9.5.1 Declaration, Definition, Call

```
int add(int, int);          /* declaration (prototype) — ends with ; */

int main(void) {
    int s = add(3, 4);      /* call — 3,4 are actual arguments */
    printf("%d", s);        /* 7 */
    return 0;
}

int add(int a, int b) {     /* definition — a,b are formal parameters */
    return a + b;
}
```

- A function can **return only one value** (via `return`); `void` = returns nothing.
- Default return type in old C = `int`; C99 onwards a declaration is required.
- Functions cannot be defined inside other functions in standard C (**no nesting of definitions**).
- **Library functions** (`printf`) vs **user-defined functions**.

9.5.2 Call by Value vs Call by Reference ★

Call by Value (default in C)	Call by Reference (via pointers in C; `&` reference in C++)
A **copy** of the argument is passed	The **address** of the argument is passed
Changes inside the function do **not** affect the caller's variable	Changes **do** affect the caller's variable
`swap(a, b)` with `void swap(int x, int y)` — no swap happens	`swap(&a, &b)` with `void swap(int *x, int *y)` — swap works

```
void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }
```

9.5.3 Recursion ★

A function calling itself. Needs a **base case** to terminate, otherwise **stack overflow**.

Factorial:

```
long fact(int n) {
    if (n <= 1) return 1;        /* base case */
    return n * fact(n - 1);      /* recursive case */
}
/* fact(5) = 5*4*3*2*1 = 120 */
```

Fibonacci (0 1 1 2 3 5 8 ...):

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
/* fib(6) = 8 */
```

Recursion uses the **stack** to store each call's local variables and return address. Iteration is generally more memory-efficient; recursion is more elegant for problems like Tower of Hanoi (moves = $2^n - 1$), tree traversals, quicksort/mergesort.

9.5.4 Storage of Local Variables

- `auto` locals → **stack** (created on entry, destroyed on exit; garbage default).
- `static` locals → **data segment** (persist across calls; zero default).
- Dynamically allocated memory (`malloc` / `new`) → **heap**.
- Code → text segment; globals/static → data segment (initialised) or BSS (uninitialised).

9.6 Arrays

9.6.1 1-D Arrays

```
int a[5] = {10, 20, 30, 40, 50};   /* indices 0..4 */
```

- Collection of elements of the **same type** in **contiguous memory**.
- Index starts at **0**; last index = size − 1. C does **no bounds checking**.
- `int a[5];` on 32-bit ⇒ occupies 5 × 4 = 20 bytes; `sizeof(a)` = 20; number of elements = `sizeof(a)/sizeof(a[0])` .
- Partial initialisation `int a[5] = {1, 2};` → remaining elements = **0**.
- **Array name = address of the first element** (`a ≡ &a[0]`).

Address arithmetic: if base address of `int a[5]` is 1000 (int = 4 bytes):

`Address of a[i] = Base + i × sizeof(type)` → `a[3]` is at 1000 + 3×4 = **1012**.

9.6.2 2-D Arrays

```
int m[2][3] = {{1,2,3},{4,5,6}};   /* 2 rows, 3 columns */
```

- Stored in **row-major order** in C: 1 2 3 4 5 6.
- `Address of m[i][j] = Base + (i × ncols + j) × size` (row-major).
- In declaration `int m[][3]` the row count may be omitted, the column count may **not**.

9.6.3 Strings

A string in C = **array of characters terminated by the null character** `'\0'`.

```
char s[] = "RRB";     /* stored as 'R','R','B','\0'  → sizeof(s)=4, strlen(s)=3 */
```

- `%s` in `printf` prints characters up to (not including) `'\0'`.
- `scanf("%s", s)` stops at whitespace; `gets` / `fgets` read a full line.

9.6.4 string.h Functions ★

Function	Purpose	Example
`strlen(s)`	Length **excluding** `'\0'`	`strlen("RRB JE")` = 6
`strcpy(d, s)`	Copy s into d	—
`strcat(d, s)`	Append s to end of d	`strcat("RRB","JE")` → "RRBJE"
`strcmp(a, b)`	Compare (ASCII): returns **0 if equal**, negative if a<b, positive if a>b	`strcmp("abc","abd")` < 0
`strrev(s)`	Reverse (non-standard/Turbo C)	—
`strlwr/strupr`	Case change (non-standard)	—
`strstr(a,b)`	Find substring b in a	—
`strchr(s,c)`	Find first occurrence of char c	—

Common output trap:

```
char s[] = "Hello\0World";
printf("%s", s);            /* Output: Hello  (stops at embedded \0) */
printf("%lu", strlen(s));   /* 5 */  /* but sizeof(s) = 12 */
```

9.7 Pointers ★

9.7.1 Basics

A pointer is a variable that **stores the address of another variable**.

```
int a = 10;
int *p = &a;      /* p holds address of a          */
printf("%d", *p); /* dereference: prints 10        */
*p = 20;          /* a becomes 20                  */
```

- `&` = address-of operator; `*` = dereference (value-at-address / indirection) operator.
- Size of any pointer on a given machine is the **same** regardless of type (4 bytes on 32-bit, 8 bytes on 64-bit).
- **NULL pointer** — points to nothing (`int *p = NULL;`); dereferencing it is undefined/crash.
- **Dangling pointer** — points to memory that has been **freed/deallocated** (e.g., after `free(p)` or returning address of a local variable).
- **Wild pointer** — uninitialised pointer.
- **void pointer** (`void *`) — generic pointer; can hold any address but must be cast before dereferencing; pointer arithmetic not allowed on it (standard C).

9.7.2 Pointer Arithmetic

`p + 1` advances by **sizeof(pointed-to type)** bytes, not by 1 byte.

```
int a[4] = {5, 10, 15, 20};
int *p = a;             /* p → a[0] */
p++;                    /* now p → a[1]; address increased by 4 */
printf("%d", *p);       /* 10 */
printf("%d", *(p + 2)); /* 20 */
```

Valid: `p+n`, `p−n`, `p2−p1` (number of elements between), comparisons. Invalid: `p1+p2`, `p*2`.

9.7.3 Pointers and Arrays

`a[i] ≡ *(a + i) ≡ *(i + a) ≡ i[a]` — all four are identical!

```
int a[3] = {1, 2, 3};
printf("%d", 1[a]);   /* prints 2 — legal though bizarre */
```

An array name is a **constant pointer**: `a++` is illegal, but `p = a; p++` is fine. Array of pointers: `int p[5];`. *Pointer to array:* `int ( p) [5];`.

9.7.4 Pointer to Pointer

```
int a = 5;
int *p = &a;
int **q = &p;
printf("%d", **q);   /* 5 */
```

9.7.5 Dynamic Memory (C)

Function	Action
`malloc(n)`	Allocates n bytes, returns `void*`, contents **garbage**
`calloc(k, size)`	Allocates k×size bytes, **initialised to 0**
`realloc(p, n)`	Resize earlier allocation
`free(p)`	Releases memory (else **memory leak**)

9.8 Structures and Unions

```
struct student {          union data {
    int  roll;                int  i;
    char name[20];            float f;
    float marks;              char c;
};                        };
```

Structure	Union
Each member gets **its own memory**	All members **share the same memory**
Size ≥ sum of member sizes (plus padding)	Size = size of the **largest** member
All members hold valid values simultaneously	Only **one** member valid at a time
Keyword `struct`	Keyword `union`

Example: members `int i; float f; char c;` (32-bit) → structure size ≈ 4+4+1(+padding)=12 bytes typically; **union size = 4 bytes**.

- Access: `s.roll` for variables, `p->roll` (≡ `(*p).roll`) for pointers.
- Structures can be **nested**, copied by assignment, passed to and returned from functions; arrays inside are copied too.
- `typedef struct {...} STUDENT;` creates an alias.
- **Self-referential structure** — contains a pointer to its own type; foundation of **linked lists**:

```
struct node { int data; struct node *next; };
```

- **Bit-fields**: `struct { unsigned flag : 1; }` — width in bits.

9.9 File Structure (File Handling in C)

Files provide **permanent storage** on disk; all functions in `stdio.h` operate through a `FILE *` **pointer**.

9.9.1 Opening and Closing

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) { printf("Cannot open"); }
...
fclose(fp);
```

fopen modes ★

Mode	Meaning	If file doesn't exist	If file exists
`"r"`	Read	Returns **NULL** (error)	Reads from beginning
`"w"`	Write	Creates new	**Erases contents** (truncates)
`"a"`	Append	Creates new	Writes at **end**, old data safe
`"r+"`	Read + write	NULL	Both, from beginning
`"w+"`	Write + read	Creates	Truncates
`"a+"`	Append + read	Creates	Appends
`"rb" "wb" "ab"` ...	Same in **binary** mode	—	—

`fopen` returns **NULL on failure** — always check it.

9.9.2 I/O Functions

Character	String	Formatted	Block (binary)
`fgetc(fp)` / `fputc(c, fp)`	`fgets(s, n, fp)` / `fputs(s, fp)`	`fscanf(fp, ...)` / `fprintf(fp, ...)`	`fread` / `fwrite`

- **EOF** — special constant (usually **−1**) returned by `fgetc` at end of file. `feof(fp)` also tests end-of-file.

```
int c;
while ((c = fgetc(fp)) != EOF)
    putchar(c);                /* copy file to screen */
```

- **Random access:** `fseek(fp, offset, origin)` with origins `SEEK_SET` (0, beginning), `SEEK_CUR` (1), `SEEK_END` (2); `ftell(fp)` gives current position; `rewind(fp)` resets to start.

9.9.3 Text vs Binary Files

Text file	Binary file
Data stored as **ASCII characters**, human-readable	Data stored in **raw byte** (memory) form
Newline may be translated (`\r\n` ↔ `\n` on Windows)	No translation, exact bytes
Slightly larger, portable	Compact and faster, not human-readable
Modes `"r" "w" "a"`	Modes `"rb" "wb" "ab"`

9.10 C++ Additions (Object-Oriented Programming)

9.10.1 OOP Pillars

- **Class** — user-defined blueprint combining data + functions; **Object** — an instance of a class (memory is allocated when the object is created).
- **Encapsulation** — wrapping data and functions into one unit (class), with access control (`private` — default in class, `protected`, `public`).
- **Abstraction** — showing only essential features, hiding implementation.
- **Inheritance** — a class (derived) acquiring properties of another (base). **Types:** single, multiple (two or more base classes — allowed in C++, not in Java), multilevel (A→B→C), hierarchical (one base, many derived), hybrid (combination; can create the "diamond problem", solved with **virtual base classes**).
- **Polymorphism** — "one name, many forms."

Compile-time (static/early binding)	Run-time (dynamic/late binding)
Function overloading — same name, different parameter list, same class/scope	**Function overriding** — same signature redefined in derived class
Operator overloading (`+` redefined for objects)	Achieved via **virtual functions** and base-class pointers
Resolved by the compiler	Resolved at run time (vtable)

9.10.2 Constructors & Destructors

```
class Demo {
public:
    Demo()        { cout << "ctor";  }   // constructor
    Demo(int x)   { }                    // parameterised (overloadable)
    Demo(const Demo &d) { }              // copy constructor
    ~Demo()       { cout << "dtor";  }   // destructor
};
```

- Constructor: **same name as class, no return type (not even void)**, called automatically at object creation, can be overloaded, can take default arguments.
- Destructor: `~ClassName()`, **no parameters, no return type, cannot be overloaded** (only one), called automatically when the object is destroyed; runs in **reverse order** of construction.
- If none written, the compiler supplies a default constructor/destructor.

9.10.3 I/O, References, new/delete

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin  >> n;                 // extraction operator
    cout << "Value=" << n << endl;   // insertion operator
    int &r = n;                // reference — alias, must be initialised
    r = 50;                    // n becomes 50
    int *p = new int(7);       // heap allocation
    int *arr = new int[10];
    delete p;                  // release single object
    delete[] arr;              // release array
    return 0;
}
```

- `cin` / `cout` are stream **objects** in `iostream` (not functions). `endl` flushes and adds newline.
- A **reference** cannot be NULL, cannot be reseated, needs initialisation at declaration — unlike a pointer.
- `new` calls the constructor and returns a typed pointer (throws on failure); `malloc` does not call constructors and returns `void*`.
- Other C++ features: default arguments, `inline` functions, `friend` functions/classes (can access private members), `this` pointer, templates (generic programming), exception handling (`try/throw/catch`), `bool` type, `::` scope resolution operator.

9.11 Data Structures

A **data structure** is a way of organising data for efficient access and modification.

Linear (elements in sequence): array, linked list, stack, queue. **Non-linear:** tree, graph.

9.11.1 Arrays vs Linked Lists

Array	Linked List
Contiguous memory, fixed size (static)	Nodes anywhere in memory, dynamic size
Random access O(1) by index	**Sequential access O(n)**
Insertion/deletion costly O(n) (shifting)	Insertion/deletion O(1) once position known
Memory may be wasted/insufficient	Extra memory for pointer field

Linked list types:

- **Singly** — each node: `data + next`; traversal one direction; last node's next = NULL.
- **Doubly** — `prev + data + next`; traversal both directions; extra memory.
- **Circular** — last node points back to the **first** node (no NULL); can be singly or doubly circular.

9.11.2 Stack — LIFO ★

Last In, First Out. Insertion and deletion at one end only, called the **TOP**.

Operation	Meaning	Error condition
`push(x)`	Insert at top	**Overflow** (stack full)
`pop()`	Remove from top	**Underflow** (stack empty)
`peek()/top()`	Read top without removing	—

All stack operations are **O(1)**. Array implementation: `top` initialised to −1; push → `s[++top]=x`; pop → `return s[top--]`.

Applications: function-call management (recursion), **expression conversion & evaluation** (infix↔postfix/prefix), undo feature, parenthesis matching, backtracking, browser back button, reversing a string, DFS.

Notation forms of `A + B`:

Notation	Operator position	Example
Infix	Between operands	`A + B`
Prefix (Polish)	Before operands	`+ A B`
Postfix (Reverse Polish)	After operands	`A B +`

Worked example 1 — Infix → Postfix: convert `A + B * C - D`

Precedence: `*` > `+`, `-` (left-assoc).

`B * C` first → `BC`; *then* `A + BC` → `ABC+`; *then* `- D` → ***`ABC +D-`***

Worked example 2 — Infix → Postfix with brackets: `(A + B) * (C - D)`

→ `AB+` and `CD-` then multiply → **`AB+CD-*`**

Worked example 3 — Infix → Prefix: `(A + B) * C`

→ *applied to* `+AB` *and* `C` → **`+ABC`**

Worked example 4 — Evaluate postfix `5 6 2 + * 12 4 / -`:

Symbol	Action	Stack
5, 6, 2	push	5 6 2
`+`	6+2=8	5 8
`*`	5×8=40	40
12, 4	push	40 12 4
`/`	12/4=3	40 3
`-`	40−3=**37**	37

Answer: 37.

9.11.3 Queue — FIFO ★

First In, First Out. Insertion (**enqueue**) at the **REAR**, deletion (**dequeue**) at the **FRONT**. Both O(1).

- **Simple/linear queue** — array version wastes freed front space.
- **Circular queue** — rear wraps around: `rear = (rear + 1) % SIZE`; full when `(rear+1)%SIZE == front`; solves the wasted-space problem.
- **Deque** (double-ended queue) — insertion and deletion at **both** ends. Variants: input-restricted (insert one end only), output-restricted (delete one end only).
- **Priority queue** — element with highest priority served first (implemented efficiently by a **heap**).

Applications: CPU/printer job scheduling, buffering (keyboard, IO), BFS, call-centre systems.

9.11.4 Trees

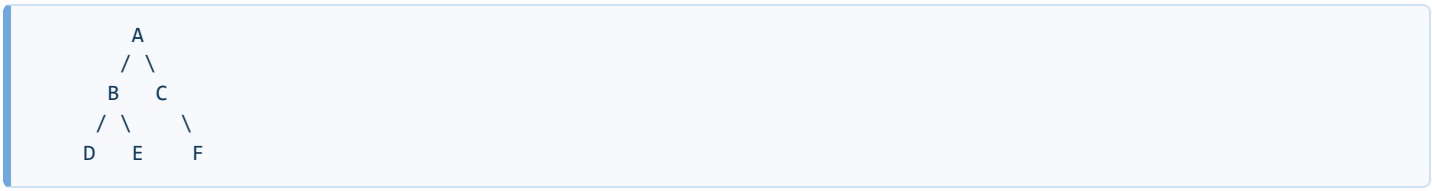
A **tree** is a non-linear hierarchical structure: a **root**, and nodes connected by edges with **no cycles**. Node terms: parent, child, **leaf** (no children), siblings, **height/depth**, degree (number of children).

Binary tree: each node has **at most 2 children** (left, right). Maximum nodes at level l = 2^l (root level 0); maximum nodes in tree of height h = 2^(h+1) − 1. A **complete** binary tree fills levels left to right; a **full/strict** tree has 0 or 2 children per node.

Traversals ★ (L = left subtree, N = node, R = right subtree):

Traversal	Order
Inorder	L → N → R
Preorder	N → L → R
Postorder	L → R → N
Level order	Level by level (uses a **queue**)

Worked example. Tree:



- **Inorder** (LNR): D B E A C F
- **Preorder** (NLR): A B D E C F
- **Postorder** (LRN): D E B F C A

Binary Search Tree (BST): left subtree keys < node key < right subtree keys. **Inorder traversal of a BST gives keys in sorted (ascending) order.** Search/insert/delete average O(log n), worst O(n) (skewed tree). Balanced variants: AVL, Red-Black.

Heap (one-liner): a complete binary tree where every parent ≥ children (**max-heap**) or ≤ children (**min-heap**); root holds max/min; used in **heapsort** and priority queues; array-based (children of index i at 2i+1, 2i+2).

9.11.5 Graphs (one-liners)

A graph G = (V, E) — set of **vertices** and **edges**; may be directed/undirected, weighted/unweighted, cyclic/acyclic. Representations: **adjacency matrix** ($O(V^2)$ space) and **adjacency list**.

- **BFS (Breadth-First Search)** — explores level by level; uses a **queue**; finds shortest path in unweighted graphs.
- **DFS (Depth-First Search)** — explores as deep as possible before backtracking; uses a **stack** (or recursion).
- A **tree is a connected acyclic graph** with exactly V−1 edges.

9.11.6 Searching

Algorithm	Requirement	Best	Average/Worst	Idea
Linear search	None (unsorted OK)	O(1)	**O(n)**	Check each element in turn
Binary search	**Sorted** data	O(1)	**O(log n)**	Compare middle, halve the range: `mid = (low+high)/2`

Maximum comparisons in binary search over n elements ≈ $\lceil \log_2(n+1) \rceil$ (e.g., 1024 elements → ~10 comparisons).

9.11.7 Sorting — Algorithms & Complexity Table ★

Algorithm	Best	Average	Worst	Space	Stable?	Method (one line)
Bubble sort	O(n)*	$O(n^2)$	$O(n^2)$	O(1)	Yes	Repeatedly swap adjacent out-of-order pairs; largest "bubbles" to the end each pass
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Select the minimum of the unsorted part, swap into place; exactly n−1 swaps
Insertion sort	**O(n)**	$O(n^2)$	$O(n^2)$	O(1)	Yes	Insert each element into its correct place in the sorted left part; best for nearly-sorted data
Quick sort	O(n log n)	**O(n log n)**	**$O(n^2)$**	O(log n)	No	Partition around a **pivot**; divide & conquer; worst case on already-sorted input with bad pivot
Merge sort	O(n log n)	O(n log n)	**O(n log n)**	**O(n)**	Yes	Divide into halves, sort, **merge**; guaranteed n log n; good for linked lists/external sorting
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Build max-heap; repeatedly extract root

*with early-exit swapped-flag optimisation.

Big-O growth order (slow-growing → fast): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$.

Bubble sort passes on n elements = n−1; comparisons total = n(n−1)/2.

9.12 DBMS (Database Management System)

9.12.1 Data, Information, DBMS

- **Data** — raw, unorganised facts (e.g., 85, "Ravi").

- **Information** — processed, meaningful data (e.g., "Ravi scored 85%").
- **Database** — organised collection of related data.
- **DBMS** — software to create, store, retrieve, and manage databases (Oracle, MySQL, SQL Server, PostgreSQL, DB2).

Advantages over file systems: controlled **redundancy**, **consistency**, **data sharing**, **security** (access control), **integrity** constraints, **backup & recovery**, program–data **independence**, concurrent access. Disadvantages: cost, complexity, need for trained staff.

9.12.2 Three-Level (ANSI-SPARC) Architecture ★

Level	Also called	Describes
External level	View level	Individual user views (highest level)
Conceptual level	Logical level	What data is stored and relationships — whole-database schema
Internal level	Physical level	**How** data is physically stored (files, indexes — lowest level)

Data independence: *logical* (change conceptual schema without changing external views) and *physical* (change internal storage without changing conceptual schema).

9.12.3 Data Models

- **Hierarchical** — tree (parent–child), e.g., IBM IMS.
- **Network** — graph, records with multiple parents (CODASYL).
- **Relational** — data in **tables (relations)**; proposed by **E. F. Codd (1970)**; the dominant model.
- **Object-oriented**, **ER model** (design tool).

9.12.4 Relational Model Terms ★

Term	Meaning	Common name
Relation	Table	Table
Tuple	A **row** of the table	Record
Attribute	A **column**	Field
Domain	Set of permitted values for an attribute	—
Degree	Number of **attributes (columns)**	—
Cardinality	Number of **tuples (rows)**	—

9.12.5 Keys ★

Key	Definition
Super key	Any set of attributes that uniquely identifies a tuple
Candidate key	A **minimal** super key (no redundant attribute)
Primary key	The candidate key chosen by the designer; **unique + NOT NULL**; one per table
Alternate key	Candidate keys not chosen as primary
Foreign key	Attribute in one table referring to the **primary key of another** table — enforces **referential integrity**
Composite key	Key made of two or more attributes

9.12.6 SQL Basics ★

SQL (Structured Query Language) — standard language of relational databases; **not case-sensitive** for keywords.

Category	Full form	Commands
DDL	Data Definition Language	`CREATE, ALTER, DROP, TRUNCATE, RENAME`
DML	Data Manipulation Language	`SELECT, INSERT, UPDATE, DELETE`
DCL	Data Control Language	`GRANT, REVOKE`
TCL	Transaction Control Language	`COMMIT, ROLLBACK, SAVEPOINT`

- `DELETE` (DML) removes rows, can use WHERE, can be rolled back; `TRUNCATE` (DDL) removes **all** rows fast, no WHERE; `DROP` removes the **table itself** including structure.

SELECT syntax:

```
SELECT column1, column2          -- * = all columns
FROM   table_name
WHERE  condition                 -- row filter (=, <>, >, LIKE, IN, BETWEEN, AND/OR)
GROUP  BY column                 -- grouping for aggregates
HAVING group_condition           -- filter on groups (used with GROUP BY)
ORDER  BY column ASC|DESC;       -- sorting (ASC is default)
```

Examples:

```
SELECT name, marks FROM student WHERE marks > 80 ORDER BY marks DESC;
SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 5;
INSERT INTO student VALUES (1, 'Ravi', 85);
UPDATE student SET marks = 90 WHERE roll = 1;
DELETE FROM student WHERE roll = 1;
```

Aggregate functions: `COUNT, SUM, AVG, MAX, MIN`. `WHERE` filters rows **before** grouping; `HAVING` filters **after**. `DISTINCT` removes duplicates. `LIKE 'A%'` — names starting with A (`%` = any string, `_` = one character).

9.12.7 Normalization ★

Normalization organises data to **minimise redundancy** and avoid insertion, update, and deletion **anomalies**, by decomposing tables.

- **1NF (First Normal Form):** every attribute contains only **atomic (indivisible) values** — no repeating groups or multi-valued cells. A table with a "phone numbers" cell holding two numbers violates 1NF; splitting into separate rows/tables fixes it.
- **2NF:** the table is in 1NF **and** every non-key attribute is **fully functionally dependent on the whole primary key** — no **partial dependency** on part of a composite key. E.g., in (StudentID, CourseID → CourseName), CourseName depends only on CourseID, violating 2NF; move it to a Course table.
- **3NF:** the table is in 2NF **and** has **no transitive dependency** — no non-key attribute depends on another non-key attribute. E.g., Student → DeptID → DeptName means DeptName transitively depends on Student; move DeptName into a Department table.
- **BCNF** (Boyce–Codd NF): stricter 3NF — every determinant must be a candidate key.

9.12.8 ER Model Basics

Proposed by **Peter Chen (1976)** as a design tool. Components and diagram symbols:

Concept	Symbol	Example
Entity (real-world object)	**Rectangle**	STUDENT
Attribute	**Ellipse/oval**	name, roll
Relationship	**Diamond**	ENROLLS
Key attribute	Ellipse with **underlined** text	roll
Multivalued attribute	**Double ellipse**	phone
Derived attribute	**Dashed ellipse**	age (from DOB)
Weak entity	**Double rectangle**	—

Relationship cardinalities: **1:1, 1:N, M:N**.

9.12.9 ACID Properties ★

Properties every **transaction** must satisfy:

Property	Meaning
A — Atomicity	All operations of a transaction happen, or **none** do ("all or nothing")
C — Consistency	Database moves from one valid state to another valid state
I — Isolation	Concurrent transactions do not interfere; intermediate results invisible to others
D — Durability	Once committed, changes are **permanent**, surviving crashes

---

9.13 Predict-the-Output: 15 Solved Programs ★★★

(Assume `#include <stdio.h>` and `int main()` wrapping each snippet; 32-bit int.)

Program 1 — Post vs pre increment

```
int a = 5;
printf("%d %d %d", a++, a, ++a);
```

Note: evaluation order of printf arguments is compiler-dependent (typically right-to-left in exams/Turbo C): `++a` →6, `a` →6, `a++` →6 (then 7). **Typical exam answer: 6 6 6.** Concept tested: side effects + unspecified order. Safe version:

```
int a = 5;
int b = a++ + ++a;   /* 5 + 7 = 12, a = 7 */
printf("%d %d", a, b);
```

Output: `7 12` — `a++` yields 5 (a→6), then `++a` makes a=7 and yields 7; 5+7=12.

Program 2 — Integer division & modulus

```
int x = 17, y = 4;
printf("%d %d %d", x / y, x % y, x / y * y + x % y);
```

Output: `4 1 17` — 17/4 truncates to 4; 17%4 = 1; 4×4+1 = 17 (identity a = (a/b)*b + a%b).

Program 3 — Bitwise operators

```
int a = 12, b = 10;
printf("%d %d %d %d", a & b, a | b, a ^ b, a << 1);
```

Output: `8 14 6 24` — 1100 & 1010 = 1000 (8); OR = 1110 (14); XOR = 0110 (6); 12<<1 = 24.

Program 4 — Ternary operator

```
int a = 10, b = 20, c;
c = (a > b) ? a * 2 : b / 2;
printf("%d", c);
```

Output: `10` — condition 10>20 false → c = 20/2 = 10.

Program 5 — switch fallthrough

```
int i = 1;
switch (i) {
    case 1: printf("One ");
    case 2: printf("Two ");  break;
    case 3: printf("Three ");
    default: printf("Def");
}
```

Output: `One Two` — matches case 1, falls into case 2 (no break after case 1), then `break` exits.

Program 6 — Short-circuit evaluation

```
int i = 0, j = 1;
if (i++ && j++)
    printf("Inside ");
printf("%d %d", i, j);
```

Output: `1 1` — `i++` yields 0 (false) so `&&` short-circuits: `j++` never runs; i became 1, j stays 1; "Inside" not printed.

Program 7 — while with post-decrement

```
int i = 5;
while (i--)
    printf("%d ", i);
```

Output: `4 3 2 1 0` — condition tests old value then decrements: tests 5 prints 4 … tests 1 prints 0, tests 0 → exit.

Program 8 — Semicolon after for

```
int i, s = 0;
for (i = 1; i <= 5; i++);
    s += i;
printf("%d %d", i, s);
```

Output: `6 6` — the `;` makes the loop body empty; loop ends with i=6; then s += i runs **once** → s=6.

Program 9 — static local variable

```
void fun(void) {
    static int c = 0;
    c++;
    printf("%d ", c);
}
int main(void) { fun(); fun(); fun(); return 0; }
```

Output: `1 2 3` — `static` c initialised once and persists across calls (an `auto` c would print `1 1 1`).

Program 10 — Recursion (factorial trace)

```
int f(int n) {
    if (n == 0) return 1;
    return n * f(n - 1);
}
/* in main: */ printf("%d", f(4));
```

Output: `24` — f(4)=4·f(3)=4·3·f(2)=4·3·2·f(1)=4·3·2·1·f(0)=24.

Program 11 — Array and pointer arithmetic

```
int a[5] = {2, 4, 6, 8, 10};
int *p = a;
printf("%d %d %d", *p + 1, *(p + 1), *(a + 4));
```

Output: `3 4 10` — `p+1` = *2+1 = 3 (dereference binds first);* `(p+1)` = a[1] = 4; `*(a+4)` = a[4] = 10.

Program 12 — String with strlen vs sizeof

```
char s[] = "RRB JE";
printf("%lu %lu", (unsigned long)strlen(s), (unsigned long)sizeof(s));
```

Output: `6 7` — strlen counts characters before `'\0'` (R,R,B,space,J,E = 6); sizeof includes the `'\0'` → 7. *(Needs `string.h`.)*

Program 13 — strcmp result

```
printf("%d", strcmp("abc", "abc") == 0 ? 1 : 0);
```

Output: `1` — strcmp returns 0 for equal strings, so the ternary yields 1.

Program 14 — Call by value vs reference

```
void byval(int x)  { x = 100; }
void byref(int *x) { *x = 100; }
int main(void) {
    int a = 5, b = 5;
    byval(a);
    byref(&b);
    printf("%d %d", a, b);
    return 0;
}
```

Output: `5 100` — byval changes only a copy; byref changes the caller's variable through its address.

Program 15 — Union memory sharing

```
union u { int i; char c; };
int main(void) {
    union u v;
    v.i = 65;
    printf("%c %d", v.c, (int)sizeof(v));
    return 0;
}
```

Output: `A 4` — members share memory; low byte of 65 is ASCII 'A' (little-endian); union size = largest member = sizeof(int) = 4.

Bonus Program 16 — Comma operator

```
int x;
x = (3, 4, 5);
printf("%d", x);
```

Output: `5` — the comma expression's value is the rightmost operand. (But `int x = 3, 4, 5;` would be a declaration error, and `x = 3, 4, 5;` without parentheses assigns 3 because `=` binds tighter than `,`.)

9.14 PYQ Corner — Actual CBT-2 Questions (22-04-2025, Electronics & Allied)

Q1. Which of the following operators is used to perform bitwise OR operation in C?

(a) `||` (b) `&` (c) `|` (d) `^`

Answer: (c) `|`.

Explanation: `|` compares operands bit by bit and sets a result bit to 1 if **either** bit is 1 (e.g., `12 | 10 = 14`). Distinguish carefully: `||` is **logical** OR (works on truth values, returns 0/1), `&` is bitwise AND, `^` is bitwise XOR.

Q2. If a C program contains only a single function, then that function must be _____.

(a) `printf()` (b) `scanf()` (c) any user-defined function (d) `main()`

Answer: (d) `main()`.

Explanation: Program execution always begins at `main()`; the linker searches for it as the entry point. A C program without `main()` cannot form an executable, so if only one function exists, it has to be `main()`. `printf` / `scanf` are library functions, not entry points.

Q3. Which of the following is a unary operator in C?

(a) `+` (addition) (b) `&&` (c) `--` (d) `<=`

Answer: (c) `--`.

Explanation: Unary operators need **one** operand: `++ -- ! ~ sizeof & *(dereference)` and unary `+/-`. Binary `+`, logical `&&`, and relational `<=` each need two operands. The only ternary operator in C is `?:`.

Q4. The correct syntax of the conditional (ternary) operator in C is:

(a) `expr1 : expr2 ? expr3` (b) `expr1 ? expr2 : expr3` (c) `expr1 ? expr2 ? expr3` (d) `expr1 : expr2 : expr3`

Answer: (b) `expr1 ? expr2 : expr3`.

Explanation: If `expr1` evaluates non-zero (true), the whole expression takes the value of `expr2`; otherwise `expr3`. Example: `max = (a > b) ? a : b;`. It is a shorthand for a simple if–else and takes three operands.

Q5. What is the output of the following code?

```
int i = 1;
while (i <= 10)
    printf("%d", i);
```

(a) 1 2 3 ... 10 (b) 10 (c) 1 printed once (d) 1 printed infinitely (infinite loop)

Answer: (d) infinite loop printing 1.

Explanation: The loop body never modifies `i`, so the condition `1 <= 10` remains true forever — the program prints `1` endlessly. The fix is to add `i++;` inside the body. This "missing increment/update" pattern is the single most repeated trick in RRB computer questions.

9.15 Practice MCQs (30 Questions)

Q1. C language was developed by:

(a) Bjarne Stroustrup (b) Dennis Ritchie (c) James Gosling (d) Ken Thompson

Q2. The number of keywords in ANSI C is:

(a) 30 (b) 32 (c) 48 (d) 64

Q3. Which of the following is an invalid identifier?

(a) `_sum` (b) `sum1` (c) `1sum` (d) `Sum_1`

Q4. The format specifier used to print a hexadecimal integer is:

(a) `%o` (b) `%h` (c) `%x` (d) `%d`

Q5. Default initial value of a `static` variable in C is:

(a) garbage (b) 0 (c) 1 (d) NULL

Q6. `printf("%d", 9/2 + 9%2);` prints:

(a) 4.5 (b) 5 (c) 4 (d) 6

Q7. Which operator has the **lowest** precedence in C?

(a) `=` (b) `,` (c) `||` (d) `?:`

Q8. `int a = 5; printf("%d", a++ + a++);` (typical exam evaluation) gives:

(a) 10 (b) 11 (c) 12 (d) 13

Q9. `printf("%d", 5 & 3);` prints:

(a) 1 (b) 7 (c) 0 (d) 8

Q10. `x << 3` is equivalent to:

(a) x × 3 (b) x × 8 (c) x ÷ 8 (d) x + 3

Q11. Which loop executes its body **at least once**?

(a) for (b) while (c) do–while (d) none

Q12. The `switch` expression in C can be of type:

(a) float (b) double (c) int/char (d) string

Q13. `continue` statement inside a loop:

(a) exits the loop (b) skips to the next iteration (c) exits the program (d) restarts the loop from the first iteration

Q14. Output of:

```
int i;
for (i = 0; i < 3; i++)
    if (i == 1) continue;
    else printf("%d", i);
```

(a) 012 (b) 02 (c) 12 (d) 0

Q15. In call by value:

(a) address is passed (b) a copy of the value is passed (c) reference is passed (d) nothing is passed

Q16. Value of `fact(0)` returned by a standard recursive factorial function is:

(a) 0 (b) 1 (c) undefined (d) −1

Q17. If base address of `int a[10]` is 2000 (int = 4 bytes), the address of `a[5]` is:

(a) 2005 (b) 2010 (c) 2020 (d) 2040

Q18. `strlen("RAILWAY")` returns:

(a) 7 (b) 8 (c) 6 (d) 9

Q19. `strcmp(s1, s2)` returns 0 when:

(a) s1 > s2 (b) s1 < s2 (c) s1 equals s2 (d) both strings are empty only

Q20. A pointer that points to a memory location that has been freed is called:

(a) NULL pointer (b) void pointer (c) dangling pointer (d) far pointer

Q21. `a[i]` is exactly equivalent to:

(a) `(a + i)` *(b)* `&a + i` *(c)* `a + i` (d) `a + *i`

Q22. Size of `union { int i; char c; double d; }` on a system with double = 8 bytes is:

(a) 13 (b) 8 (c) 4 (d) 1

Q23. Which fopen mode **erases** an existing file's contents?

(a) "r" (b) "a" (c) "w" (d) "r+"

Q24. `fgetc()` returns _____ at end of file:

(a) 0 (b) NULL (c) EOF (d) '\0'

Q25. Function overloading is an example of:

(a) run-time polymorphism (b) compile-time polymorphism (c) inheritance (d) encapsulation

Q26. A destructor in C++:

(a) can be overloaded (b) takes parameters (c) has no return type and no parameters (d) must be called explicitly always

Q27. Which data structure works on the LIFO principle?

(a) queue (b) stack (c) linked list (d) tree

Q28. The postfix form of `A * (B + C)` is:

(a) `ABC+` *(b)* `AB+C` (c) `ABC+` *(d)* `A+BC`

Q29. Inorder traversal of a Binary Search Tree yields keys in:

(a) random order (b) descending order (c) ascending order (d) level order

Q30. Which of the following is a DDL command in SQL?

(a) SELECT (b) INSERT (c) UPDATE (d) CREATE

Answer Key with Brief Explanations

Q	Ans	Explanation
1	b	Dennis Ritchie, Bell Labs, 1972 (Stroustrup → C++, Gosling → Java)
2	b	ANSI C has 32 reserved keywords
3	c	Identifier cannot **start with a digit**
4	c	`%x` = hex, `%o` = octal, `%d` = decimal
5	b	static/extern default to 0; auto/register hold garbage
6	b	9/2 = 4 (integer), 9%2 = 1; 4+1 = 5
7	b	Comma has lowest precedence; `=` is second-lowest
8	b	5 + 6 = 11 (first a++ yields 5 then a=6, second yields 6) — the standard exam convention
9	a	101 & 011 = 001 = 1
10	b	Left shift by n multiplies by 2^n → x×8
11	c	do–while is exit-controlled: body first, test after
12	c	switch works on integral types only (int/char/enum)
13	b	continue skips rest of body, moves to next iteration
14	b	prints 0 and 2; i=1 skipped by continue
15	b	A copy is passed; original unchanged
16	b	Base case: 0! = 1
17	c	2000 + 5×4 = 2020
18	a	R-A-I-L-W-A-Y = 7 characters (\0 not counted)
19	c	0 = equal; negative/positive indicate ASCII ordering
20	c	Dangling pointer → freed/deallocated memory
21	a	Array indexing is defined as `*(a+i)`
22	b	Union size = size of largest member = 8 (double)
23	c	"w" truncates existing file; "a" preserves and appends
24	c	EOF (typically −1) signals end of file
25	b	Overloading resolved at compile time (static binding); overriding via virtual functions is run-time

Q	Ans	Explanation
26	c	`~Class()` — no arguments, no return type, cannot be overloaded
27	b	Stack = Last In First Out; queue = FIFO
28	a	(B+C) → `BC+` ; then `A * BC+` → operands A and BC+ followed by * → `ABC+*`
29	c	BST property ⇒ inorder gives sorted ascending sequence
30	d	CREATE/ALTER/DROP/TRUNCATE = DDL; SELECT/INSERT/UPDATE/DELETE = DML

> **Note on Q28:** option (b) `AB+C` *would correspond to* `(A+B) C` — a common distractor. Trace bracket-first: `(B+C)` → `BC+` , then multiply by A → `ABC+*` .

9.16 Rapid-Revision Sheet (last-day flip)

A. Operator Precedence (top → bottom)

```
() []  ->  .  postfix++/--          (highest)
prefix++/--  !  ~  unary+/-  *  &  sizeof  (cast)   [R→L]
*  /  %
+  -
<<  >>
<  <=  >  >=
==  !=
&      ^      |         (AND before XOR before OR)
&&     ||
?:                       [R→L]
=  +=  -=  ...           [R→L]
,                        (lowest)
```

One-liners: only ternary = `?:` · only ones needing lvalue = `++ -- = &` · `sizeof` is an operator, not a function · logical ops give 0/1; bitwise work on bits.

B. Format Specifiers & Escape Sequences

`%d` int · `%u` unsigned · `%ld` long · `%f` float · `%lf` double · `%c` char · `%s` string · `%o` octal · `%x` hex · `%p` address · `%%` percent sign

`\n` newline · `\t` tab · `\0` null · `\\` backslash · `\a` bell · `\b` backspace · `\r` carriage return

C. Sizes (32-bit) & Storage Classes

char 1 · short 2 · int 4 · long 4 · float 4 · double 8 · any pointer 4

auto → stack, garbage · register → CPU register, no `&` · static → persists, 0 · extern → global, 0

D. Complexity Table

Structure/Algo	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked list	O(n)	O(n)	O(1)*	O(1)*
Stack/Queue op	—	—	O(1)	O(1)
BST (avg)	—	O(log n)	O(log n)	O(log n)

*at known position. Linear search O(n) · Binary search O(log n), needs **sorted** data.

Sorting: Bubble/Selection/Insertion **$O(n^2)$** · Quick avg **O(n log n)** worst $O(n^2)$ · Merge & Heap **O(n log n)** always · Merge needs O(n) extra space · Stable: bubble, insertion, merge.

E. Data-Structure Flip Summary

DS	Principle	Insert/Delete point	Uses
Stack	**LIFO**	Top only (push/pop)	Recursion, undo, infix→postfix, DFS, parenthesis matching
Queue	**FIFO**	Rear in / Front out	Scheduling, buffering, BFS
Circular queue	FIFO	rear = (rear+1)%SIZE	Reuses freed space
Deque	Both ends	Front & rear	Both stack + queue behaviour
Singly LL	Sequential	Anywhere via pointers	Dynamic memory lists
Circular LL	Last → first	—	Round-robin scheduling
Binary tree	Hierarchical	—	Traversals: In = LNR, Pre = NLR, Post = LRN
BST	left < root < right	O(log n) avg	**Inorder = sorted**
Heap	Complete tree, parent ≥/≤ child	Root extract	Heapsort, priority queue
Graph	V, E	—	BFS uses **queue**, DFS uses **stack**

F. SQL Command Classification

DDL: CREATE · ALTER · DROP · TRUNCATE · RENAME

DML: SELECT · INSERT · UPDATE · DELETE

DCL: GRANT · REVOKE **TCL:** COMMIT · ROLLBACK · SAVEPOINT

Keys: Primary = unique + NOT NULL · Foreign = refers to another table's primary key · Candidate = minimal super key

Row = tuple · Column = attribute · #columns = degree · #rows = cardinality

ACID = Atomicity · Consistency · Isolation · Durability

1NF atomic values → 2NF no partial dependency → 3NF no transitive dependency → BCNF determinant = candidate key

Three levels: External (view) — Conceptual (logical) — Internal (physical)

G. Ten Facts They Keep Asking

1. Bitwise OR = `|`; logical OR = `||`.

2. Single-function C program ⇒ that function is `main()`.
3. `--`, `++`, `!`, `~`, `sizeof` are **unary**; `?:` is the only **ternary**.
4. Ternary syntax: `expr1 ? expr2 : expr3`.
5. Loop without an update of the control variable = **infinite loop**.
6. `do-while` executes at least once; `while` / `for` may execute zero times.
7. Missing `break` in switch ⇒ **fallthrough**.
8. String ends in `'\0'`; `strlen` excludes it, `sizeof` includes it.
9. Stack = LIFO (push/pop at top); Queue = FIFO (rear-in, front-out).
10. Inorder traversal of a BST gives **sorted output**; postfix of `(A+B)C` = `AB+C`.

End of Chapter 9. Next: Chapter 10 — Basic Electronics Engineering (as per Annexure-VII C sequencing of your book plan).